

Chapter 3 – Operator & Ekspresi

Dalam pemrograman, komputer tidak bisa “berpikir” sendiri.

Kita harus memberi tahu **apa yang harus dihitung, apa yang harus dibandingkan**, dan Untuk itulah kita menggunakan **operator** dan **ekspresi**.

Pada chapter ini, Anda akan belajar:

- Apa itu operator dan ekspresi
- Cara melakukan perhitungan dengan Python
- Cara membandingkan nilai
- Cara menggabungkan beberapa kondisi
- Cara menulis ekspresi sederhana dalam program

Tujuan Pembelajaran

Setelah mempelajari chapter ini, Anda diharapkan mampu:

- Memahami konsep operator dan ekspresi
- Menggunakan operator aritmatika
- Menggunakan operator perbandingan
- Menggunakan operator logika
- Menggunakan operator penugasan
- Menggabungkan operator dalam ekspresi sederhana

3.1 Apa Itu Operator dan Ekspresi?

Bayangkan sebuah kalkulator. - Tombol +, -, ×, ÷ adalah **operator**

- Perhitungan seperti $10 + 5$ adalah **ekspresi** Dalam Python:

```
hasil = 10 + 5
```

Pada contoh di atas: - + adalah **operator** - $10 + 5$ adalah **ekspresi** - hasil akan berisi nilai **15**

Jadi: - **Operator** = alat untuk melakukan operasi - **Ekspresi** = gabungan nilai + operator yang menghasilkan sebuah nilai

Contoh lain:

```
x = 7
y = 3
z = x * y + 2
```

Ekspresi di atas adalah $x * y + 2$, dan hasil akhirnya disimpan ke dalam z.

3.2 Operator Aritmatika

Operator aritmatika digunakan untuk melakukan perhitungan matematika.

Operator	Contoh	Keterangan
+	$a + b$	Penjumlahan

Operator	Contoh	Keterangan
-	a - b	Pengurangan
*	a * b	Perkalian
/	a / b	Pembagian (hasil float)
%	a % b	Sisa pembagian (modulus)

Contoh penggunaan:

```
a = 10
b = 3
print(a + b) # 13
print(a - b) # 7
print(a * b) # 30
print(a / b) # 3.3333333333333335
print(a % b) # 1
```

Operator % (**modulus**) sering dipakai untuk mengecek: - Apakah suatu bilangan habis dibagi? - Apakah bilangan **genap** atau **ganjil**?

```
angka = 10
print(angka % 2 == 0) # True, karena 10 adalah bilangan genap
```

3.3 Operator Perbandingan

Operator perbandingan digunakan untuk **membandingkan dua nilai**. Hasilnya selalu berupa nilai **Boolean**: True atau False.

Operator	Arti	Contoh	Hasil
==	Sama dengan	5 == 5	True
!=	Tidak sama dengan	5 != 3	True
>	Lebih besar	5 > 3	True
<	Lebih kecil	5 < 3	False
>=	Lebih besar atau sama	5 >= 5	True
<=	Lebih kecil atau sama	3 <= 5	True

Contoh:

```
a = 10
b = 7
print(a == b) # False
print(a > b) # True
print(a <= b) # False
```

Operator ini sangat penting saat kita membuat **keputusan** dalam program.

3.4 Operator Logika

Operator logika digunakan untuk **menggabungkan beberapa kondisi sekaligus**. Tiga operator logika utama di Python:

```
and  
or  
not
```

Contoh:

```
umur = 15  
print(umur > 10 and umur < 20)  
print(umur < 10 or umur > 18)  
print(not (umur == 15))
```

Penjelasan: - and → **semua** kondisi harus True - or → **minimal satu** kondisi bernilai True - not → membalik nilai (True menjadi False, dan sebaliknya)

Contoh dengan kalimat:

```
nilai = 80  
absen_baik = True  
lulus = (nilai >= 75) and absen_baik  
print(lulus) # True
```

Artinya: "Siswa dinyatakan lulus jika **nilai minimal 75** dan **absen baik**."

3.5 Menggabungkan Operator dalam Ekspresi

Dalam program nyata, kita sering menggabungkan **operator aritmatika, perbandingan, dan logika** dalam satu ekspresi.

Contoh:

```
nilai = 75  
lulus = nilai >= 70 and nilai <= 100  
print(lulus)
```

Langkah berpikirnya: - nilai >= 70 → True - nilai <= 100 → True - True and True → True

Maka variabel lulus bernilai True.

Contoh lain:

```
umur = 16  
memiliki_ktp = False  
boleh_mengemudi = umur >= 17 and memiliki_ktp  
print(boleh_mengemudi) # False
```

Ini meniru cara manusia berpikir: “Boleh mengemudi jika **umur minimal 17 tahun** dan **sudah punya KTP.**”

::: {`.tip title=“Urutan Evaluasi (Prioritas Operator)”`} Seperti di matematika, Python juga memiliki **urutan prioritas** operator. Secara umum:

Tanda kurung () Operator aritmatika (*, /, +, -, dll.) Operator perbandingan (>, <, ==, dll.) Operator logika (not, and, or) Gunakan **kurung ()** untuk memperjelas maksud ekspresi dan menghindari kebingungan. :::

3.6 Operator Penugasan (Assignment)

Operator penugasan digunakan untuk **memberi nilai** pada variabel.

Bentuk dasar:

```
x = 10
```

Python juga menyediakan bentuk singkat yang menggabungkan operasi aritmatika dengan penugasan:

```
x = 10
x += 5    # sama dengan x = x + 5 → x menjadi 15
x -= 3    # sama dengan x = x - 3 → x menjadi 12
x *= 2    # sama dengan x = x * 2 → x menjadi 24
x /= 4    # sama dengan x = x / 4 → x menjadi 6.0
```

Kelebihan penggunaan bentuk singkat ini: - Kode lebih **ringkas** - Lebih **mudah dibaca** - Sering digunakan dalam perulangan dan pemrosesan data

3.7 Operator Keanggotaan (Membership Operators)

Operator keanggotaan digunakan untuk mengecek apakah suatu nilai terdapat di dalam kumpulan data (seperti list atau string).

Dua operator utama:

in not in

Contoh:

```
buah = ["apel", "jeruk", "mangga"]
print("apel" in buah)      # True
print("pisang" in buah)    # False
print("durian" not in buah) # True
```

Artinya: “apel” in buah → “Apakah ‘apel’ ada di dalam daftar buah?” “durian” not in buah → “Apakah ‘durian’ **tidak** ada di dalam daftar buah?”

3.8 Operator dan Ekspresi dalam Program Nyata

Operator dalam Python tidak berdiri sendiri: operator selalu bekerja di dalam sebuah ekspresi, dan hasil ekspresi itu dapat:

Disimpan ke dalam variabel Langsung dicetak dengan print() Digunakan sebagai bagian dari kondisi if, while, dan sebagainya Contoh:

```
x = 10
hasil = (x + 5) * 2
print("Hasil:", hasil)           # Menggunakan hasil ekspresi
print(hasil > 20)                # Menggunakan ekspresi dalam perbandingan
print(hasil > 20 and hasil < 40) # Menggabungkan beberapa operator
```

Dengan memahami cara kerja operator dan ekspresi, Anda akan lebih mudah menulis **logika program** yang benar dan jelas.

3.9 Latihan

Tugas: Buat dua variabel a dan b. Lakukan operasi berikut:

- Penjumlahan
- Perkalian
- Perbandingan $a > b$
- Tampilkan semua hasilnya ke layar.

Contoh solusi:

```
a = 8
b = 5
print("a + b =", a + b)
print("a * b =", a * b)
print("a > b =", a > b)
```

Cobalah mengubah nilai a dan b, lalu perhatikan bagaimana hasil ekspresi ikut berubah.

Pada chapter berikutnya, Anda akan mempelajari **Struktur Kontrol Alur**, yaitu cara membuat **keputusan** dan **perulangan** dalam program Python.